Universidad Don Bosco

Facultad de Ingeniería

Escuela de Ingeniería en Computación

Programación Estructurada

Docente: Nombre del docente

Artículo Científico (ciclo 2025)

Actividad #2 (25 %)

Procesamiento y almacenamiento de datos distribuidos

Análisis del artículo Bigtable: A Distributed Storage System for Structured Data (Chang et al., 2006)

Proyecto de Cátedra: MediControl SV

Integrantes del equipo

Integrante 1: Nombre completo – Carné

Integrante 2: Nombre completo – Carné

Integrante 3: Nombre completo – Carné

Fecha de entrega: miércoles 07 de octubre de 2026

Índice

1. Datos Generales del Artículo....3

2. Resumen del Artículo....3

2.1 ¿Cuál es el objetivo del artículo?....3

2.2 ¿Qué problema busca resolver?....3

2.3 ¿Qué metodología utiliza?....4

Figura 1. Arquitectura general de Bigtable y sus componentes....6

2.4 ¿Cuáles son los principales hallazgos o conclusiones?....6

3. Análisis Crítico....7

3.1 ¿Qué aporta este artículo al campo del análisis y diseño de sistemas y bases de datos?....7

3.2 ¿Qué conceptos del curso se relacionan con el contenido del artículo?....7

3.3 ¿Qué ventajas y desventajas se presentan en el enfoque del artículo?....8

3.4 ¿Estamos de acuerdo con los resultados o el planteamiento? ¿Por qué?....8

4. Aplicación Práctica....9

4.1 ¿Qué mejoras sugeriríamos al enfoque planteado en el artículo?....9

4.2 ¿Cómo podríamos aplicar las ideas del artículo en nuestro Proyecto de Cátedra?....10

Figura 2. Arquitectura propuesta para MediControl SV con componentes distribuidos....12

Figura 3. Modelo entidad-relación simplificado de MediControl SV....12

5. Conclusión Personal....14

5.1 ¿Qué aprendimos del artículo?....14

5.2 ¿Nos cambió la forma de pensar sobre algún tema visto en este curso?....14

5.3 Cierre....15

6. Referencias....15

1. Datos Generales del Artículo

Para esta actividad seleccionamos el artículo *Bigtable: A Distributed Storage System for Structured Data*, publicado por un equipo de ingenieros de Google en 2006. Lo elegimos porque trata directamente el tema que nos corresponde, el procesamiento y almacenamiento de datos distribuidos, y porque su contenido se conecta con varios objetivos del curso: estudiar modelos de bases de datos usados en contextos reales, comparar el enfoque tradicional (relacional) con uno contemporáneo y evaluar cómo una arquitectura afecta el rendimiento de un sistema.

Dato	Detalle
Título del artículo	Bigtable: A Distributed Storage System for Structured Data
Autor(es)	Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes y Robert E. Gruber (Google, Inc.)
Año de publicación	2006
Revista o conferencia	7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06), Seattle, Washington, noviembre de 2006. Páginas 205–218. El trabajo recibió el reconocimiento de mejor artículo (Best Paper) de la conferencia.
Link / URL	https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/
Tema del curso	Procesamiento y almacenamiento de datos distribuidos

2. Resumen del Artículo

2.1 ¿Cuál es el objetivo del artículo?

El objetivo del artículo es presentar Bigtable, un sistema de almacenamiento distribuido diseñado para administrar datos estructurados a una escala muy grande: petabytes de información repartidos en miles de servidores de hardware común (Chang et al., 2006). Los autores describen el modelo de datos que ofrece el sistema, cómo está construido y cómo se comporta en producción. Además, buscan demostrar que un diseño relativamente simple y flexible puede servir a aplicaciones muy distintas entre sí, como el índice de búsqueda web, Google Earth, Google Finance, Google Analytics y las búsquedas personalizadas.

2.2 ¿Qué problema busca resolver?

El problema de fondo es que las aplicaciones de Google manejaban volúmenes de datos que superaban lo que un solo servidor, o una base de datos relacional convencional, podía almacenar y procesar de forma eficiente y económica. A esto se sumaba que las aplicaciones tenían necesidades muy diferentes. Algunas guardan datos pequeños, como

direcciones web, y otras guardan datos enormes, como páginas completas o imágenes satelitales. Unas necesitan procesamiento masivo por lotes y otras requieren respuestas rápidas para usuarios en tiempo real.

Los autores explican que Bigtable se parece a una base de datos en muchos aspectos (escalabilidad, rendimiento, confiabilidad), pero ofrece una interfaz distinta y no soporta un modelo relacional completo. En lugar de eso, da a los clientes control directo sobre cómo se organizan y almacenan los datos. En otras palabras, el problema que se resuelve es cómo crecer horizontalmente, agregando máquinas, sin perder disponibilidad ni rendimiento.

2.3 ¿Qué metodología utiliza?

Se trata de un artículo de diseño e implementación de sistemas. La metodología combina la descripción de la arquitectura con una evaluación experimental y la experiencia de uso real. Podemos resumirla en los siguientes pasos:

1. **Definición del modelo de datos.** Bigtable es un mapa ordenado, disperso, distribuido y persistente en varias dimensiones. Cada valor se identifica con tres coordenadas: una clave de fila, una clave de columna y una marca de tiempo. El valor es una cadena de bytes sin interpretar, de modo que cada aplicación decide qué guarda en ella.
2. **Selección de bloques de construcción.** En lugar de construir todo desde cero, Bigtable se apoya en servicios ya existentes: el sistema de archivos distribuido GFS (Ghemawat et al., 2003) para guardar datos y registros, el formato de archivo SSTable para los datos ordenados e inmutables, y el servicio de bloqueos Chubby (Burrows, 2006) para coordinar a los servidores.
3. **Diseño de la implementación.** El sistema se compone de una biblioteca cliente, un servidor maestro y muchos servidores de tablets. Las tablas se dividen en rangos de filas llamados tablets, que se reparten entre los servidores. El maestro asigna tablets y balancea la carga; los servidores atienden las lecturas y escrituras.
4. **Refinamientos de rendimiento.** Se agregan mejoras como grupos de localidad (agrupar columnas que se leen juntas), compresión, cachés de dos niveles, filtros de Bloom para evitar lecturas innecesarias en disco y una estrategia eficiente para el registro de escrituras.
5. **Evaluación y casos reales.** Se miden lecturas y escrituras, secuenciales y aleatorias, aumentando el número de servidores, y se describen aplicaciones reales que usan el sistema.
6. **Lecciones aprendidas.** Los autores cierran con las enseñanzas de operar el sistema durante varios años.

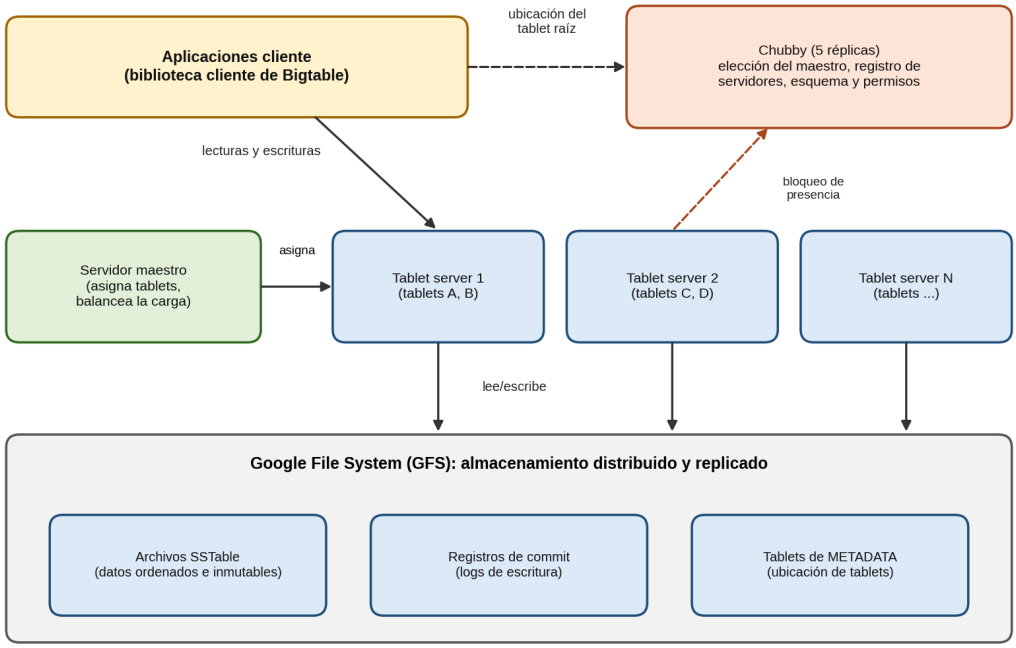
A continuación se explican con más detalle las piezas principales. El modelo de datos de Bigtable se puede entender con la siguiente tabla, que resume sus elementos:

Elemento	Descripción
Fila (row)	Se identifica con una clave de texto. Los datos se mantienen ordenados por esa clave. Las lecturas y escrituras sobre una misma fila son atómicas, sin importar cuántas columnas toque la operación.
Familia de columnas	Grupo de columnas relacionadas. Es la unidad de control de acceso y de contabilidad de uso. Las columnas dentro de una familia se pueden crear dinámicamente.
Marca de tiempo	Permite guardar varias versiones del mismo dato. Cada familia define cuántas versiones conservar o por cuánto tiempo, y el sistema elimina las antiguas automáticamente.
Tablet	Rango continuo de filas. Es la unidad de distribución y de balanceo de carga: cada tablet la atiende un servidor y se divide cuando crece demasiado.

El artículo ilustra el modelo con el ejemplo de una tabla llamada Webtable, que guarda páginas web. Como clave de fila se usa la dirección web con el nombre del dominio invertido (por ejemplo, com.cnn.www), lo que deja juntas en disco las páginas de un mismo sitio. Una familia de columnas guarda el contenido de la página en distintas versiones según la fecha de captura, y otra guarda los textos de los enlaces que apuntan hacia ella.

Para ubicar los datos, Bigtable usa una jerarquía de tres niveles parecida a un árbol B+: un archivo en Chubby apunta a un tablet raíz, este apunta a los tablets de una tabla especial llamada METADATA, y estos indican dónde está cada tablet de usuario. Los clientes guardan en caché esas ubicaciones, por lo que normalmente no tienen que recorrer toda la jerarquía. Para las escrituras, cada servidor primero anota la operación en un registro de commit (guardado en GFS) y luego la aplica a una estructura en memoria llamada memtable. Cuando la memtable crece, se convierte en un archivo SSTable (compactación menor), y periódicamente se combinan varios archivos en uno solo (compactación mayor) para mantener las lecturas rápidas. La Figura 1 muestra los componentes y cómo se relacionan.

Figura 1. *Arquitectura general de Bigtable y sus componentes*



Nota. Elaboración propia con base en Chang et al. (2006).

2.4 ¿Cuáles son los principales hallazgos o conclusiones?

- **Escalabilidad.** En las pruebas de rendimiento, el volumen total de operaciones por segundo aumentó en más de cien veces al pasar de 1 a 500 servidores. Los autores aclaran que el crecimiento no fue perfectamente lineal, y que se nota una caída en el rendimiento por servidor al sumar máquinas, sobre todo en las lecturas aleatorias.
- **Uso real a gran escala.** Cuando se publicó el artículo, Bigtable operaba en cientos de clústeres de producción (unos 388) con aproximadamente 24 500 servidores de tablets en total, lo que respalda que el diseño funciona fuera del laboratorio.
- **Flexibilidad del modelo.** Una misma tecnología sirvió para cargas por lotes y de baja latencia, con tamaños de dato muy distintos, sin tener que rediseñar el sistema para cada aplicación.
- **Eficiencia por refinamientos.** Las mejoras de compresión, caché y filtros de Bloom redujeron el espacio ocupado y las lecturas a disco. Para Webtable, los autores reportan una compresión de alrededor de diez a uno.
- **Integración con el procesamiento distribuido.** Bigtable puede usarse como fuente de entrada y destino de salida de trabajos MapReduce (Dean & Ghemawat, 2004), lo que une el almacenamiento con el procesamiento masivo de datos.
- **Lecciones.** Los sistemas distribuidos grandes fallan de muchas formas, no solo por caídas de red o de máquinas: también por errores de memoria, relojes

desincronizados, mantenimiento y fallos en otros sistemas de los que se depende. Por eso recomiendan no agregar funciones hasta saber cómo se usarán, monitorear bien el sistema y preferir diseños simples.

3. Análisis Crítico

3.1 ¿Qué aporta este artículo al campo del análisis y diseño de sistemas y bases de datos?

Consideramos que el artículo hace cuatro aportes importantes. El primero es mostrar un camino práctico para almacenar datos a escala masiva: en lugar de depender de un servidor cada vez más potente (escalamiento vertical), reparte la información entre muchas máquinas comunes (escalamiento horizontal) y asume que algunas van a fallar. El segundo es proponer un modelo de datos distinto al relacional, basado en filas ordenadas, familias de columnas y versiones por tiempo, que se adapta bien a datos dispersos y cambiantes.

El tercero es la forma de diseñar el sistema: separar responsabilidades y reutilizar servicios especializados (almacenamiento en GFS, coordinación en Chubby, datos en SSTables) hace que cada pieza sea más simple de entender y de mantener. Y el cuarto es que el trabajo influyó en toda una generación de bases de datos NoSQL de columnas anchas, como Apache HBase y Apache Cassandra, por lo que entenderlo ayuda a comprender tecnologías que se usan hoy en la industria (Kleppmann, 2017).

3.2 ¿Qué conceptos del curso se relacionan con el contenido del artículo?

Concepto del curso	Relación con el artículo
Modelo de datos	Bigtable propone un modelo alternativo al relacional (mapa ordenado de fila, columna y tiempo), lo que permite comparar enfoques tradicionales y contemporáneos.
Normalización y desnormalización	En Bigtable no hay uniones (joins): los datos se organizan según cómo se van a consultar, aceptando cierta duplicación a cambio de rapidez. Es lo contrario a la normalización estricta.
Llaves e índices	La clave de fila funciona como índice principal porque mantiene los datos ordenados. Elegir bien esa clave (como invertir el dominio en Webtable) es una decisión de diseño clave. Los filtros de Bloom son una técnica adicional de búsqueda.
Particionamiento	Los tablets dividen las tablas por rangos de filas y se reparten entre servidores. Es un ejemplo de fragmentación horizontal.
Replicación y disponibilidad	Los datos se guardan en GFS, que mantiene varias copias, y Chubby se mantiene con cinco réplicas. Si un servidor cae, sus tablets pasan a otro.
Transacciones y concurrencia	Solo se garantiza atomicidad por fila; no hay transacciones entre varias filas. Esto contrasta con las propiedades ACID de un sistema relacional.

Concepto del curso	Relación con el artículo
Bitácora y recuperación	El registro de commit y la memtable permiten recuperar las escrituras recientes si un servidor falla, igual que el registro de transacciones de un gestor de bases de datos.
Requisitos no funcionales	El diseño responde a requisitos de escalabilidad, latencia, disponibilidad y costo, que en el análisis de sistemas se definen antes de diseñar.
Arquitectura de sistemas	Cliente, maestro y servidores de tablets forman una arquitectura por capas, con responsabilidades separadas.
Procesamiento distribuido	La integración con MapReduce muestra cómo el almacenamiento y el procesamiento se complementan al trabajar con grandes volúmenes.

3.3 ¿Qué ventajas y desventajas se presentan en el enfoque del artículo?

Ventajas	Desventajas
Escala agregando máquinas comunes, sin depender de equipos costosos.	No ofrece SQL, uniones ni un modelo relacional completo; las consultas complejas deben resolverse en la aplicación.
Modelo flexible: las columnas se agregan dinámicamente y los datos dispersos no desperdician espacio.	Las transacciones se limitan a una sola fila, lo que complica operaciones que afectan varios registros a la vez.
Guarda versiones de los datos por marca de tiempo, útil para historiales.	Exige diseñar con cuidado la clave de fila y los patrones de acceso desde el inicio; un mal diseño afecta el rendimiento.
Alta disponibilidad: reasigna tablets si un servidor falla y se apoya en almacenamiento replicado.	Depende de otros servicios (GFS y Chubby); si Chubby deja de estar disponible, el sistema puede dejar de atender datos.
Optimizaciones probadas: grupos de localidad, compresión, cachés y filtros de Bloom.	Es un sistema interno de Google; no se puede usar directamente fuera de esa empresa, solo alternativas inspiradas en él.
Funciona para cargas por lotes y en tiempo real, y se integra con MapReduce.	Operarlo es complejo: requiere monitoreo, personal especializado y pruebas ante muchos tipos de fallo.
Respaldado por años de uso en productos reales y con resultados medidos.	Para sistemas pequeños es más complejidad de la necesaria. Además, el artículo no profundiza en seguridad, cifrado ni privacidad de datos.

3.4 ¿Estamos de acuerdo con los resultados o el planteamiento? ¿Por qué?

En general estamos de acuerdo con el planteamiento, con algunos matices. Coincidimos en que, para volúmenes de datos muy grandes y con muchos usuarios, repartir la información y aceptar fallos como algo normal es una decisión realista. Los resultados además son creíbles porque no se limitan a pruebas de laboratorio: el artículo presenta

mediciones y también casos de uso reales con datos de producción, y reconoce abiertamente los límites del sistema, como que el rendimiento por servidor baja al crecer el clúster.

Sin embargo, no creemos que Bigtable sea una solución universal ni un reemplazo de las bases de datos relacionales. Su propio diseño renuncia a las uniones y a las transacciones completas para ganar escala. En el teorema CAP (Brewer, 2012) siempre hay que decidir qué se sacrifica ante una falla de red, y este sistema prioriza la consistencia dentro de cada fila. Para una aplicación como un sistema clínico, donde importa que una receta y la consulta que la originó se guarden juntas o no se guarden, las propiedades ACID de un motor relacional siguen siendo muy valiosas. Sistemas posteriores, como Google Spanner, intentaron justamente combinar escala con transacciones más fuertes, lo que confirma esa limitación. Por eso valoramos el artículo sobre todo como fuente de ideas y principios de diseño más que como una receta para copiar.

4. Aplicación Práctica

4.1 ¿Qué mejoras sugeriríamos al enfoque planteado en el artículo?

Aunque el artículo es un trabajo muy sólido, identificamos siete mejoras que, a nuestro criterio, lo harían más completo, sobre todo si se piensa en su uso en áreas sensibles como la salud:

1. **Seguridad y privacidad desde el diseño.** El control de acceso se aplica a nivel de familia de columnas, pero se podría ampliar con cifrado de datos almacenados y en tránsito, y con registros de auditoría de quién accedió a qué. Esto evitaría depender de parches posteriores.
2. **Transacciones entre varias filas.** Permitir, aunque sea de forma opcional, operaciones atómicas sobre más de una fila haría el sistema útil para casos como registrar una consulta y su receta en una sola operación.
3. **Capa de consulta e índices secundarios.** Incorporar un lenguaje de consultas parecido a SQL y soporte nativo de índices secundarios reduciría el trabajo que hoy recae en cada aplicación y facilitaría su adopción por equipos con formación relacional.
4. **Evaluación con cargas más variadas.** Complementar las pruebas con cargas mixtas de lectura y escritura, mediciones de latencia en los casos más lentos (percentiles altos) y datos de costo operativo daría una imagen más completa del comportamiento real.
5. **Guías para el diseño de claves.** Incluir recomendaciones claras y ejemplos para elegir la clave de fila y evitar puntos calientes, es decir, rangos de filas que reciben demasiado tráfico, ayudaría a quienes están aprendiendo a usar este tipo de sistemas.

6. **Observabilidad y pruebas de fallos.** Los autores mencionan la importancia del monitoreo; se podría formalizar con paneles de métricas estándar y pruebas periódicas que provoquen fallos a propósito para comprobar que la recuperación funciona.
7. **Interoperabilidad y estándares.** Facilitar la integración con otros sistemas mediante interfaces abiertas y formatos comunes reduciría la dependencia de una sola tecnología y facilitaría migraciones futuras.

4.2 ¿Cómo podríamos aplicar las ideas del artículo en nuestro Proyecto de Cátedra?

Nuestro Proyecto de Cátedra se llama **MediControl SV**. Es un sistema web para la gestión integral de un consultorio médico en El Salvador. Permite administrar pacientes, médicos, especialidades, horarios y citas desde una sola plataforma. Los médicos gestionan el expediente clínico, las consultas, los signos vitales, las recetas, los estudios y los seguimientos, y los pacientes consultan sus citas, recetas, resultados y otra información autorizada a través de un portal. El propósito es mejorar la organización, seguridad, trazabilidad y eficiencia del consultorio mediante una plataforma centralizada.

Es importante ser realistas: un consultorio no tiene el volumen de datos de Google y no necesita un sistema como Bigtable. Aun así, los principios del artículo sí se pueden aplicar a una escala más pequeña, usando una base de datos relacional (por ejemplo, MySQL o PostgreSQL) y servicios comunes de la nube. Estas son las ideas que adoptaríamos:

a) Particionamiento y diseño de claves. Así como Bigtable divide las tablas en tablets por rangos de filas, en MediControl SV las tablas que más crecen (consultas, signos vitales, bitácora) se pueden particionar por año. También se crearían índices compuestos sobre los campos que se consultan juntos, como paciente y fecha, y se evitarían claves que concentren todo el tráfico en un solo lugar.

b) Versiones con marca de tiempo. Bigtable no sobrescribe el dato anterior: guarda versiones. Aplicamos la misma idea al expediente clínico: cuando se corrige un diagnóstico o una dosis, se conserva el valor anterior con fecha, hora y usuario que hizo el cambio. Así se mantiene un historial completo y se cumple el objetivo de trazabilidad del proyecto. Los registros clínicos importantes no se borrarían físicamente, solo se marcarían como inactivos.

c) Familias de columnas y grupos de localidad. Bigtable agrupa los datos que se leen juntos y separa los pesados. De forma parecida, en MediControl SV los datos de uso frecuente (datos del paciente, citas, recetas) vivirían en la base de datos, mientras que los archivos pesados (imágenes de estudios, PDF de resultados) irían a un almacén de archivos independiente, referenciado desde la base. Esto mantiene rápidas las consultas habituales.

d) Replicación y disponibilidad. Siguiendo la idea de GFS de guardar varias copias, usaríamos una base de datos principal para las escrituras y una réplica para lecturas, como las

del portal del paciente y los reportes. Si el servidor principal falla, la réplica puede asumir su lugar. Además, los respaldos automáticos se guardarían en otra ubicación distinta al servidor principal y se probaría periódicamente su restauración.

e) Registro de escritura y recuperación. Del mismo modo que Bigtable usa un registro de commit para recuperar datos tras una falla, activaríamos el registro de transacciones del motor de base de datos y la recuperación a un punto en el tiempo, para poder restaurar el sistema justo antes de un error.

f) Caché. Bigtable usa cachés para evitar lecturas repetidas a disco. En nuestro proyecto, datos que casi no cambian, como el catálogo de especialidades, los horarios publicados y la lista de medicamentos, se guardarían en una caché en memoria para responder más rápido y aliviar la base de datos.

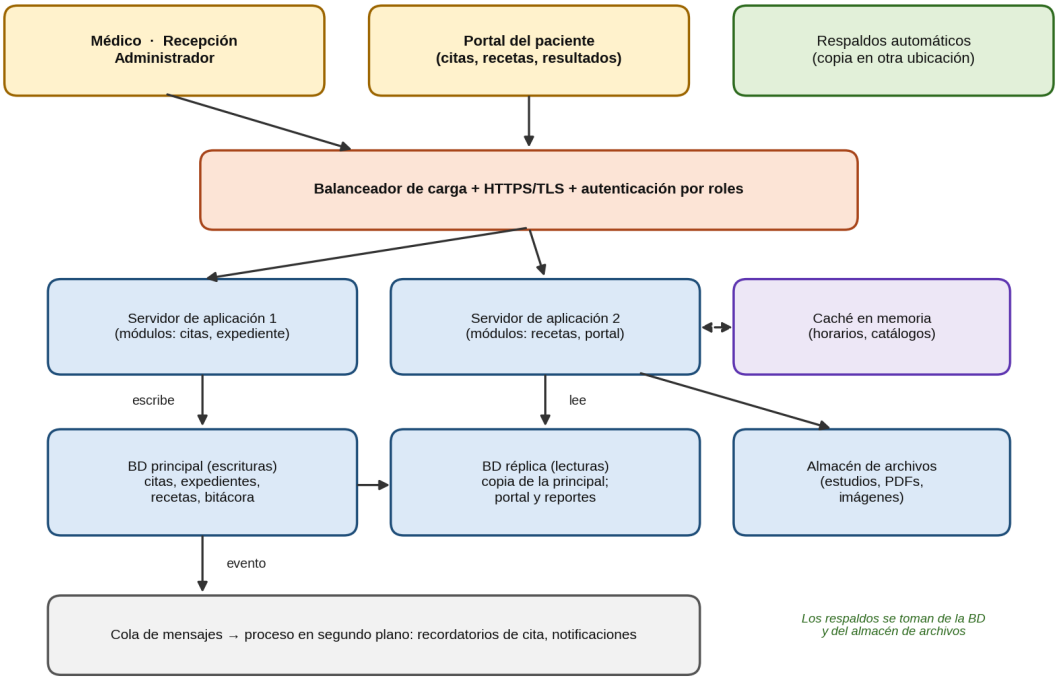
g) Procesamiento asíncrono y por lotes. La relación entre Bigtable y MapReduce nos recuerda que no todo debe hacerse en el momento en que el usuario hace clic. Los recordatorios de citas, las notificaciones y los reportes estadísticos (por ejemplo, consultas por especialidad al mes) se procesarían en segundo plano mediante una cola de mensajes y tareas programadas, sin hacer esperar al personal del consultorio.

h) Coordinación de citas. Chubby se usa para coordinar mediante bloqueos. En MediControl SV, la regla de que un médico no pueda tener dos citas en el mismo horario se garantizaría con una restricción única en la base de datos sobre médico, fecha y hora, más una transacción al reservar, de modo que dos personas no puedan tomar el mismo espacio al mismo tiempo.

i) Seguridad y confidencialidad. Aquí cubrimos una de las debilidades que señalamos del artículo. Se definirían roles diferenciados (administrador, recepción, médico y paciente), de forma que cada quien vea solo lo que le corresponde: el médico accede a los expedientes de sus pacientes, el paciente solo a su propia información y recepción gestiona citas sin ver el contenido clínico. Se usarían contraseñas almacenadas con hash seguro, conexión cifrada (HTTPS), cierre de sesión por inactividad y una bitácora que registre todos los accesos. Todo esto se diseñaría respetando el secreto médico y la normativa salvadoreña aplicable en materia de protección de datos personales.

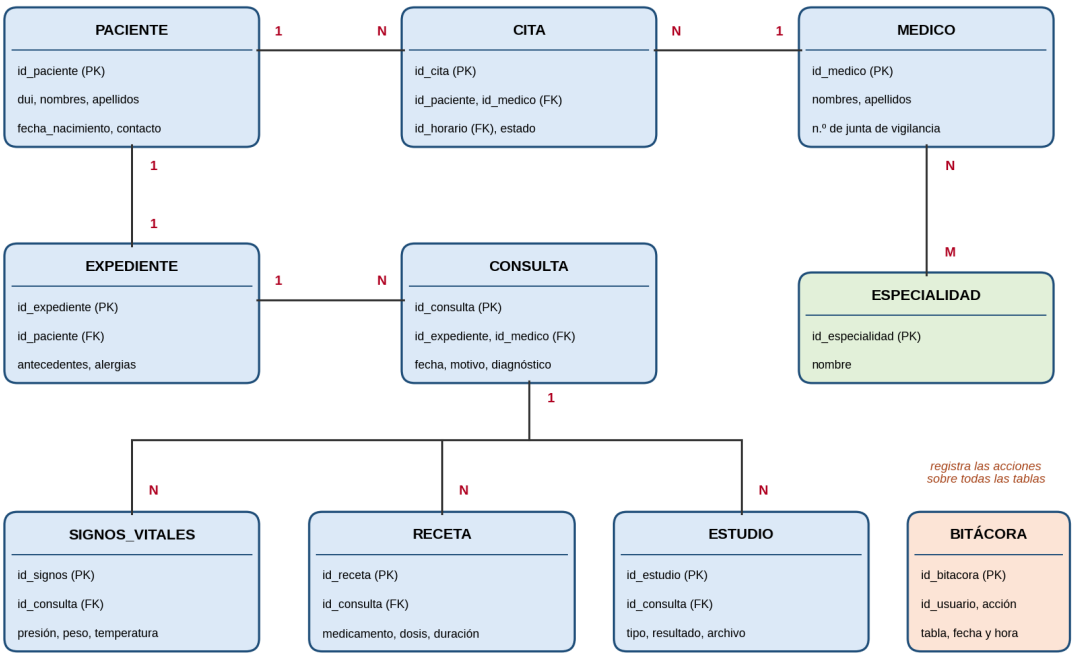
La Figura 2 resume la arquitectura que proponemos para MediControl SV y la Figura 3 muestra el modelo de datos simplificado, con las entidades principales y sus relaciones.

Figura 2. *Arquitectura propuesta para MediControl SV con componentes distribuidos*



Nota. Elaboración propia con base en las ideas de Chang et al. (2006).

Figura 3. *Modelo entidad-relación simplificado de MediControl SV*



Nota. Elaboración propia. PK: llave primaria; FK: llave foránea.

La siguiente tabla resume cómo se traduce cada idea del artículo en una decisión concreta para el proyecto:

Idea del artículo	Aplicación en MediControl SV	Resultado esperado
Tablets (particionamiento)	Tablas históricas particionadas por año; índices compuestos paciente-fecha	Consultas rápidas aunque crezca el historial
Versiones por marca de tiempo	Historial de cambios del expediente y de las recetas	Trazabilidad completa
Familias de columnas	Datos frecuentes en la base; archivos pesados en un almacén aparte	Menor tiempo de respuesta
GFS (replicación)	Base principal con réplica de lectura y respaldos en otra ubicación	Disponibilidad y recuperación ante fallos
Registro de commit	Registro de transacciones y recuperación a un punto en el tiempo	Datos recuperables
Cachés	Caché en memoria para horarios y catálogos	Menor carga en la base de datos
MapReduce (procesos por lotes)	Cola de mensajes para recordatorios, notificaciones y reportes	Sistema ágil para el usuario
Chubby (coordinación)	Restricción única y transacción al reservar citas	Cero citas duplicadas

Para llevar esto a la práctica, proponemos un plan por fases:

1. **Fase 1, análisis.** Levantar requisitos funcionales y no funcionales con el consultorio, definir casos de uso y reglas de negocio (por ejemplo, la duración de las citas y quién puede ver cada dato).
2. **Fase 2, diseño.** Elaborar el modelo entidad-relación, normalizarlo hasta tercera forma normal, decidir qué tablas se particionan y diseñar la arquitectura y los prototipos de pantallas.
3. **Fase 3, desarrollo.** Construir primero los módulos de pacientes, médicos y citas, luego expediente, recetas y estudios, y por último el portal del paciente y los procesos en segundo plano.
4. **Fase 4, pruebas.** Realizar pruebas funcionales, de seguridad y de carga con datos de ejemplo, además de simular la caída del servidor principal para comprobar la recuperación.
5. **Fase 5, puesta en marcha.** Capacitar al personal, migrar los expedientes en papel y monitorear el sistema durante las primeras semanas.

Para saber si estas decisiones funcionan, propondríamos medir los siguientes indicadores. Los valores son metas planteadas por nuestro equipo y se ajustarían con el consultorio.

Indicador	Meta propuesta
Tiempo de respuesta al buscar un paciente o una cita	Menos de 2 segundos en el uso normal
Disponibilidad del sistema en horario de atención	99.5 % o más
Pérdida máxima de datos ante una falla (respaldo)	No más de 24 horas, idealmente menos
Tiempo máximo para restaurar el servicio	Menos de 4 horas
Citas duplicadas en un mismo horario	Cero
Accesos al expediente registrados en la bitácora	100 %

5. Conclusión Personal

5.1 ¿Qué aprendimos del artículo?

Aprendimos que guardar y procesar datos en muchos servidores no es simplemente "tener más espacio". Implica decidir cómo se reparten los datos, cómo se recuperan cuando algo falla y qué se está dispuesto a sacrificar, como las uniones o las transacciones completas, para ganar velocidad y capacidad. Entendimos que cada decisión de diseño tiene un costo y un beneficio, y que no existe un sistema perfecto para todos los casos.

También aprendimos conceptos concretos que antes veíamos de forma abstracta: el particionamiento por rangos, las versiones por marca de tiempo, el registro de escrituras para la recuperación, los filtros de Bloom y la importancia de elegir bien la clave de una tabla. Ver cómo se combinan en un sistema real, medido y usado en producción, nos ayudó a entender para qué sirven. Además, aprendimos a leer un artículo científico de forma crítica, identificando su objetivo, su método, sus resultados y también sus límites.

5.2 ¿Nos cambió la forma de pensar sobre algún tema visto en este curso?

Sí. Antes pensábamos que diseñar una base de datos era, sobre todo, normalizar tablas y definir relaciones, y que el modelo relacional era la única forma correcta de hacerlo. Este artículo nos mostró que el modelo depende del problema: cuando el volumen es enorme y las consultas son predecibles, puede ser mejor organizar los datos según cómo se van a leer, aunque se pierda parte de la normalización.

También cambió nuestra visión del análisis y diseño de sistemas. Entendimos que los requisitos no funcionales, como la escalabilidad, la disponibilidad y el tiempo de respuesta,

deben pensarse al principio y no después de programar, porque determinan la arquitectura completa. Por último, nos dimos cuenta de que un sistema que funciona pero no es seguro, no se puede recuperar de una falla o no deja rastro de los cambios no es realmente confiable, sobre todo cuando maneja información de personas, como el expediente de un paciente.

5.3 Cierre

Este trabajo nos permitió conectar un artículo científico con un proyecto real. Aunque MediControl SV es mucho más pequeño que Bigtable, las ideas de particionar, replicar, versionar, guardar en caché y procesar tareas en segundo plano nos dan una guía clara para construirlo con mejor rendimiento, seguridad y orden. Nos llevamos no solo conocimiento técnico, sino una forma de pensar más responsable al diseñar sistemas que guardan información importante de las personas.

6. Referencias

Brewer, E. (2012). CAP twelve years later: How the "rules" have changed. *Computer, 45*(2), 23–29. https://doi.org/10.1109/MC.2012.37

Burrows, M. (2006). The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)* (pp. 335–350). USENIX Association.

Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., Chandra, T., Fikes, A., & Gruber, R. E. (2006). Bigtable: A distributed storage system for structured data. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)* (pp. 205–218). USENIX Association. https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/

Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI '04)* (pp. 137–150). USENIX Association.

Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google file system. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03)* (pp. 29–43). ACM. https://doi.org/10.1145/945445.945450

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.